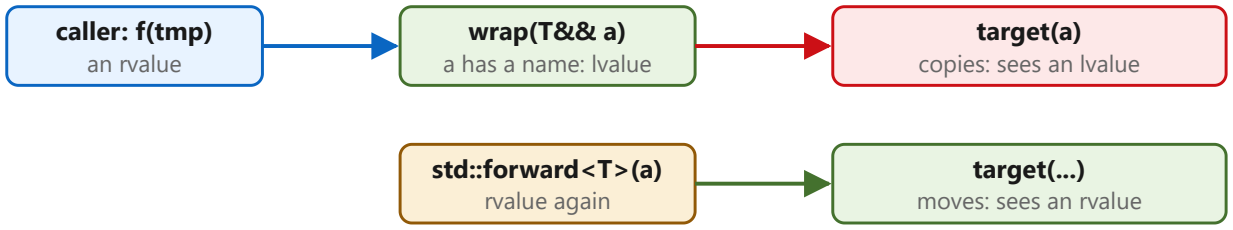
Perfect Forwarding

Passing arguments through a wrapper without changing their value category: why `std::forward` is needed, how reference collapsing makes it work, and where it fails. Assumes Sink Parameters and Move Traps and Function Templates.

Why a wrapper loses the category

Suppose you write a wrapper that passes its arguments on to another function (`make_unique`, `emplace_back`, a logging wrapper). The caller may pass an lvalue or an rvalue, and the target should copy or move accordingly. But inside the wrapper, the parameter has a **name**, so it is an lvalue, and the target would *always copy*.



std::forward restores what the caller passed.

The wrapper must pass the argument on exactly as it received it.

```
// always an rvalue
template <typename T> void wrapMove(T&& a) { f(std::move(a)); }
// the caller's category
template <typename T> void wrapFwd (T&& a) { f(std::forward<T>(a)); }

std::string t = "t";
// && : moved from t, though the caller passed an lvalue!
wrapMove(t);
wrapFwd(t);                    // const& : t stays intact
wrapFwd(std::string("u"));     // && : a temporary is moved
// prints: && const& &&   (compiled)
```

`std::forward` restores it

`std::forward<T>(a)` casts `a` back to what the caller passed: an rvalue reference if the argument was an rvalue, an lvalue reference if it was an lvalue. It works because the parameter is a **forwarding reference**: `T&&` where `T` is deduced (see Function Templates).

```
template <typename T, typename ... Args>
// forwarding references
std::unique_ptr<T> makeUnique(Args&& ... args) {
    // each argument keeps its category
    return std::unique_ptr<T>(new T(std::forward<Args>(args) ... ));
}

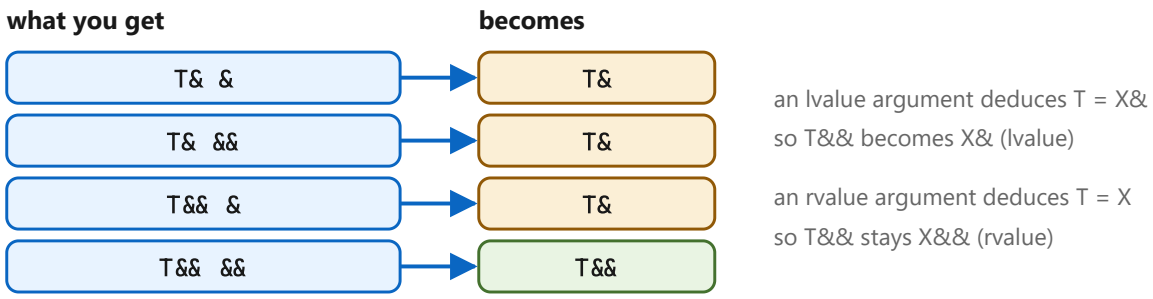
std::string s = "abc";
// lvalue → Widget's constructor COPIES it
auto a = makeUnique<Widget>(s);
// rvalue → Widget's constructor MOVES it
auto b = makeUnique<Widget>(std::string("x"));
```

	`STD::MOVE(X)`	`STD::FORWARD<T>(X)`
casts to	always an rvalue	an rvalue only if the argument was one
use on	a variable you are done with	a forwarding reference, once
when wrong	steals an lvalue the caller wanted	forgetting it makes every argument a copy

> **ONCE ONLY** Forward each parameter at most once. After `std::forward` the object may have been moved from.

Reference collapsing

Why does `T&&` sometimes mean an lvalue reference? Because C++ does not allow a reference to a reference, but templates can create one by substitution. The rule for resolving it is **reference collapsing**: if either side is an lvalue reference, the result is an lvalue reference.



A forwarding reference works because T& && collapses to T&.

ARGUMENT	DEDUCED `T`	PARAMETER `T&&` BECOMES
an lvalue `s` of type `string`	`string&`	`string&`
an rvalue `string("x")`	`string`	`string&&`

`std::forward<T>` uses exactly that: with `T = string&` it returns `string&` (an lvalue), with `T = string` it returns `string&&`.

Where forwarding fails

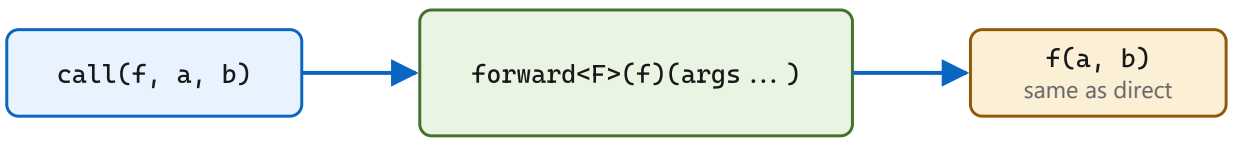
Perfect forwarding is not perfect for every argument. Deduction fails or picks the wrong type in a few cases.

ARGUMENT	WHY FORWARDING FAILS
a braced list `{1, 2}`	a braced list has no type, so `T` cannot be deduced; name the type or pass a `std::initializer_list`
`0` or `NULL` as a null pointer	deduced as `int`, which does not convert to a pointer; use `nullptr`
the name of an overloaded or template function	the compiler cannot choose which overload to deduce; cast to the exact function pointer type or use a lambda
a bit-field	a non-const reference cannot bind to a bit-field; copy it into a local first
a `static const` data member with no definition	binding a reference odr-uses it and the linker may fail

These are the cases Effective Modern C++ Item 30 lists. In practice the first two cause most surprises.

Forwarding in practice

Forwarding is how the standard library builds generic wrappers: `make_unique`, `make_shared`, `emplace_back`, `std::thread`, `std::function` and `std::invoke` all forward their arguments. Here is the shape of a wrapper that calls anything:



decltype(auto) plus forwarding returns exactly what the callee returns.

```cpp
template <typename F, typename... Args>
// forward the function and every argument
decltype(auto) call(F&& f, Args&&... args) {
    return std::forward<F>(f)(std::forward<Args>(args)...);
}

// 5: lvalue and rvalue arguments behave as in a direct call (compiled)
call(add, 2, 3);
int n = 0;
// bump takes int&: n really is incremented, so n == 1 (compiled)
call(bump, n);
```

PART	WHY
`Args&&...`	a pack of forwarding references: each argument binds to whatever it was
`std::forward<Args>(args)...`	expands once per argument, restoring each one's category
`decltype(auto)` return	returns exactly what the callee returns, including references

- If a wrapper passes an argument to **two** calls, forward only the last one.
- A constrained wrapper (a concept or `enable_if`) avoids hijacking copy construction: an unconstrained `T&&` constructor beats the copy constructor for non-const lvalues (Effective Modern C++ Item 26).

Further reading by section

Where each section of this lesson is explained in more depth. Pages are PDF viewer pages (the number your PDF reader shows).

SECTION	BOOK	PART OF THE BOOK	PDF PAGES
std::forward restores it	Effective Modern C++	Item 23, Understand std::move and std::forward	176 to 182
	Effective Modern C++	Item 25, Use std::move on rvalue references, std::forward on universal references	186 to 195
	C++ Move Semantics	9.1 to 9.3 Perfect forwarding; universal versus rvalue references	159 to 166
	C++ Move Semantics	Chapter 11 Perfect passing with auto&&	191 to 202
Reference collapsing	C++ Move Semantics	9.1 to 9.3 Perfect forwarding; universal versus rvalue references	159 to 166
Where forwarding fails	C++ Move Semantics	9.4 to 9.5 Overload resolution with universal references; perfect forwarding in lambdas	167 to 170